

21/11/25

* Reinforcement learning ^{→ Agentic AI or Autonomous application}
1) RL is a type of ML / PL where an agent learns by interacting with an environment unlike supervised learning where an agent learns pattern.
2) In RL, agent learns from experience.

* Key terminologies in RL are:-

a) Agent

- Agent is the learner.
- Agent can be a hardware based or software based entity
- Eg:- Robot, self-driving cars, game applications (AI based)

b) Environment

- It is the world made up of states with which the agent interacts.
- Eg:- Chess board, stock Market, Road route.

c) State.

- The current situation of the agent in the environment
- Eg: In a grid game 2^*2 a position like (1,1) can be a state.

d) Action $\rightarrow$ based on policy

- Action is what the agent can do in a given state to come out of it.
- Eg. In a 2^*2 grid actions can be up, down, right, left.

e) Reward

- It is a numeric score given after each action
- It can be considered as a feedback after taking an action.

f) Policy → created based on value fn

- Policy are rules or strategies that help agent in deciding action at each state.
- Agent learns policies by experiencing environment across multiple episodes.

g) Values function: → eg Bellman eqⁿ

It is a function that calculate a value of each state in an environment.

* What are the characteristics of ^{RL} AI? (enlarge)

- 1) It is more focused on goal directed learning
- 2) It interacts with environment while training in form of episodes to take enviro experience and learn policies.
- 3) RL learns how to map situations to actions.
- 4) In RL, the learner is not told which actions to take but it has to discover using RL.
- 5) Thus RL is based on trial and errors
- 6) RL can operate in both static and dynamic environments

* Explain the concept of Exploration and Exploitation
1) In RL action plays an important role to navigate from one state to another but the actions must be taken in such a way that it yields maximum rewards to the agent.

2) To do this agent must balance between two behaviours i.e. Exploration and Exploitation.

a) Exploration:-

- It is trying new actions to discover better rewards new states and hidden benefits.
- In this agent takes risks and experiments something new everytime.
- Every agent must do exploration because without exploration agent might miss better strategies better path or reward.
- Eg:- If a human always eat a favourite dish in a favourite restaurant he will never be able to taste new dishes in other better restaurants.

b) Exploitation:-

- Exploitation uses the best known action that gives maximum reward based on present knowledge.
- With exploitation the agent plays safe based on his experience and doesn't risk itself.
- Eg: If you are always eating in your favourite restaurant you are safe from other not so good restaurants.

3) There should always be a balance between exploration and exploitation.

4) This tradeoff is called exploration - exploitation dilemma.

5) This balanced approach helps agent to take good actions based on his present experience but also to explore new actions and learn over time.

Bellman eqⁿ: $Q(S, A) = Q(S, A) + \alpha [R + \gamma \max_{A'} (Q(S', A')) - Q(S, A)]$

discount factor $[0, 1]$

max value
re discount
1 means

$S' A' = (0, 0)$

3 * 3

	0	1	2
0	(0, 0)	(0, 1)	(0, 2)

next stage
value

$Q((0, 0), A)$
 state action

- 1] What is RL Learning? what are the key components of Reinforcement learning.
- 2] What are the characteristics of Reinforcement learning?
- 3] Explain the concept of Exploration and exploitation?
- 4] Compare supervised, unsupervised and Reinforcement learning.
- 5] What are the applications of Reinforcement learning?

Prac 2 : 4*4

	0	1	2	3
0	0,0	0,1	0,2	0,3
1	1,0	1,1	1,2	1,3
2	2,0	2,1	2,2	2,3
3	3,0	3,1	3,2	3,3

Rewards
 up = (-1, 0)
 down = (1, 0)
 left = (0, -1)
 right = (0, 1)

Goal state = 3,3

→ 2,1 (up)

state = 16

↳ in rows.

action in columns.

-1, 0

1, 1 → moves to (1, 1)

new_r, new_c = r + dr, c + dc → action_reward.
 (action_reward)

Eg → (2,1) → Right → (0,1) → reward add thus
 $\downarrow \quad \downarrow$
 dr dc

$\begin{matrix} 2, 1 \\ 0, 1 \\ 2, 2 \end{matrix}$ → new_r = 2 & new_c = 2 (next state)

4) Compare supervised, Unsupervised and RL

Ans

Criteria	SL	UL	RL
- Data Labels	Uses labeled data	No labels used	No fixed labels; uses reward signals.
- Goal	Predict o/p (classification/regression)	Find hidden patterns or structure	Learn best actions to maximize rewards.
- Learning style	Teacher-guided learning	Self learning	Trial-and-error learning.
- O/p type	Specific o/p	Cluster, grps, reduced dimensions	Optimal policy (best action plan)
- Feedback Type	Correct answer given	No feedback	Reward or penalty after action
- Need of env	Not required	Not required	Required since agent interacts with environment.
- Examples	Spam detection, disease prediction, stock forecasting	Customer segmentation, anomaly detection	Robotics, gaming, autonomous vehicles
- Where used	NLP, finance, healthcare, recommendation systems	Market analysis, pattern discovery	Industrial automation, simulations, education tech.

5) What are the applications of Reinforcement learning?

Ans: ① Robots:-

Helps robots learn how to move, pick things and do tasks on their own.

② Games:-

Used to teach computers how to play games better (like chess, racing, etc).

③ Self-Driving cars:-

Helps cars learn how to drive safely - stop, turn, avoid accidents.

④ Finance:-

used to decide when to buy or sell shares to get better profits

⑤ Healthcare:-

Helps doctors or systems choose the best treatment for patients.

⑥ Recommendation systems:-

Shows better suggestions (movie, songs, products) by learning what users like.

⑦ Industries:- Helps machine work better, faster and with fewer mistakes

⑧ Education Apps:- Learning apps use RL to change difficulty based on how the student performs.

5/12/25

Unit 2

Markov Decision Process.

Page No.	
Date	

(6m)

policies/rules

framework for defining a approach

* MDP A markov Decision process or MDP is a mathematical framework used to describe how an RL agent interacts with an environment and makes decisions to maximize rewards.

- It defines everything that is needed for RL

- (i) what the agent sees - states.
- (ii) what the agent does - actions
- (iii) How the environment responds - rewards.

present state depends on previous state only

(iv) Markov Property - The framework of MDP is made up of 5 elements states, Actions, Transitions, probabilities, reward function, discount factor (S, A, P, R, γ) .

Thus, $MDP = (S, A, P, R, \gamma)$

explanation

MDP
Deterministic
prob = 1.0 then
Have transition
prob is not 1.0 then it is 1.

- ① State(s): A finite $\emptyset$
- ② Action (A)
- ③ P (Transition Probability) :-
A probability function $P(s' | s, a)$ that defines the likelihood of transitioning from state s to s' after taking action a
- ④ Reward (R) :-
A function $R(s, a, s')$ that assigns a reward for moving from state s to s' via action a
- ⑤ γ (Discount factor) :-
A value in the range $[0, 1]$ that determines the importance of future rewards.

model-based $\rightarrow$ transition prob depends
model-free $\rightarrow$ without transition prob is
eg Q-learning $\rightarrow$ it is

Page No.	
Date	

* Compare MDP with Q-Learning: (Enlarge)

- MDP is a mathematical problem definition

- MDP

Q-learning

① Q-learning is a solⁿ method used when part of MDP is unknown.

- MDP is a mathematical problem definition

- Q-learning is a solⁿ method used when part of MDP is unknown

- So, MDP is the model of env and Q-learning is an algo to take correct actions

- Q-learning is normally used with MDP if the MDP framework is incomplete or transition probability is deterministic.

Q How do we calculate state value and action value in MDP.

Ans ^s ① state value in MDP is calculated using state value function $V(s)$. $V(s)$ tells us how good is a state, another function is action value function, $Q(s, a)$ which tells us how good is taking an action in a state.

② In Deterministic MDP Bellman Equation

$$Q(s, a) = Q(s, a) + \alpha [R + \gamma \max_{a'} (Q(s', a')) - Q(s, a)]$$

is used to calculate action value (Q -value) for every state the optimal state value is calculated using these action value as follows

$$V^*(s) = \max_a Q^*(s, a)$$

Thus, we get the optimal value for each state based on which we make optimal policies for the environment.

With the help of these optimal policies the agent can thus make correct decisions in the environment.

* Optimality Principle in MDP:-

The optimal policy has the property that whatever the initial state is, the remaining decisions must form an optimal policy with respect to the state resulting from the first decision.

Thus,

- ① ASPer optimal policy every step taken by an agent must be optimal.
- ② Today's, optimal decision must lead to tomorrow's optimal decision
- ③ There are no bad steps in an optimal policy
- ④ There is no need to evaluate paths in optimal policy rather we evaluate states

Q. Compare Deterministic and Stochastic MDP.

Q. What are the different types of MDP.

Ans - Deterministic MDP.

- Stochastic MDP.
- Finite MDP
- Infinite MDP.
- Fully observable MDP.
- Partially observable MDP.

12/12/25

Unit 1

Page No.	
Date	

Q. What are the adv and disadv of RL?

Ans. Adv:- Learns from interaction:-

RL learns by doing not by being told.

② Works in dynamic and complex environments -
RL handles environments where rule changes frequently.

③ Learn optimal long term behavior -
RL optimizes policies to learn best behavior.

④ No need of labelled data -
Unlike supervised learning, RL doesn't need labelled samples.

⑤ Can achieve superhuman performance:-
for eg: deep mines RL agent.

⑥ Works well for sequential decision making:-
RL is best when decision happens in steps.

✗ Disadvantage of RL:-

① Requires lot of data or trial.

② Expensive computation:- Training is slow and requires powerful hardware.

③ Hard to converge.

It is possible that the agent learns some suboptimal policy instead of optimal policy.

④ Reward design is difficult:-

A poorly design reward leads to wrong decision

⑤ Exploration can be dangerous

⑥ Hard to scale to real world problems:-
Real environments are noisy, unpredictable

Q7.6 What is On Policy and Off-Policy Approach?

Ans ① An On-policy method learns the value of the policy that the agent is actually following. It learns from the actions it actually takes. The Policy is used for behaviour and learning.
Eg:- SARSA (State-Action-Reward-^{Next}State-Action)
Next.

The Bellman Equation for SARSA is:-

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma Q(s',a')]]$$

Here a' is the actual next action taken by the agent

So, the update depends on the policy be followed. It is similar to the human way of learning driving skills.

For eg:- You drive a car your way and you also learn based on your actions and mistakes.

Thus, you learn based on what you do (extra don't add)

② Off-policy Approach :-

An off-policy method learns ^{the} value of a different policy than one used to generate data. It learns optimal actions even if it takes suboptimal or exploratory actions.

Thus, there are two different policies :-

- ① Behaviour Policy
- ② Target Policy

- Q-learning update for this is :-

$$Q(s, a) \leftarrow Q(s, a) + \alpha [R + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

^{learning factor}
_{max from Q-table & then sub them}

The update is thus based on Greedy approach

(For eg :- You may drive the car randomly sometimes to explore shortcuts but you are learning the best route not ^{the} one that you are currently driving.

Thus, you learn what you should do from what you actually do. ^{extra don't add}